Perceive, Plan, Act, Self-Correct: An Architectural Framework for Goal-Directed Agentic AI Systems

Hameem M. Mahdi[a,*]

[a] *Mayo Clinic, Rochester, MN, 55905, USA*

Abstract

Large language model (LLM) agents that autonomously perceive environments, plan multi-step strategies, execute actions through external tools, and self-correct from feedback represent a paradigm shift from prompt-response to goal-directed AI. Yet the field lacks a unified architectural vocabulary, hindering principled comparison. Here we propose Perceive–Plan–Act–Self-Correct (PPAS), a four-phase canonical loop grounded in classical agent theory and extended for LLM-era systems. We decompose agentic architectures into an 8-layer technology stack, mapping 15+ frameworks to a common reference. Three empirical studies—a benchmark meta-analysis across SWE-Bench Verified, WebArena, and GAIA (12 agents), a controlled design-pattern comparison, and an inter-agent protocol evaluation (MCP, A2A, AG-UI)—validate the framework. Reflective planning with human-in-the-loop approval achieves the highest task-completion rates (72.5% SWE-Bench) while reducing irreversible-action failures by 73%. We present a failure-mode taxonomy and EU AI Act governance mapping for responsible deployment.

Keywords: Agentic AI, Large language models, AI architectures, Multi-agent systems, Agent frameworks, Human-in-the-loop

1. Introduction

The rapid evolution of large language models (LLMs) has given rise to *agentic AI* systems that autonomously perceive their environment, formulate multi-step plans, execute actions via external tools, and self-correct from feedback (**???**). Unlike static generative models, these systems maintain persistent state, interact with dynamic environments, and exhibit goal-directed behaviour over extended time horizons.

This shift has been catalysed by three converging capabilities: reliable function-calling from frontier LLMs (**??**), standardised tool-integration protocols such as

*Corresponding author

Email address: `mahdi.hameem@mayo.edu` (Hameem M. Mahdi)

the Model Context Protocol (MCP) (?), and orchestration frameworks implementing planning, memory, and multi-agent coordination (??). Despite explosive growth—over 126,000 GitHub stars for LangChain alone—the field lacks a unified architectural vocabulary. Each framework introduces bespoke abstractions, creating a "framework zoo" where selecting the right architecture remains ad hoc.

Here we propose the PERCEIVE–PLAN–ACT–SELF-CORRECT (PPAS) framework, a four-phase canonical loop for agentic AI systems. We address five research questions:

RQ1. What architectural primitives distinguish agentic AI from other autonomous and generative AI paradigms?

RQ2. How does an 8-layer technology stack map to production implementations?

RQ3. Which design patterns yield the highest task-completion rates across benchmark suites?

RQ4. How do inter-agent protocols affect coordination efficiency?

RQ5. What failure modes and governance requirements apply to deployed systems?

The paper makes four contributions: (i) a formal four-phase agent loop unifying existing patterns; (ii) an 8-layer technology stack mapping 15+ frameworks; (iii) three empirical studies validating the framework; and (iv) a failure-mode taxonomy with EU AI Act governance mapping. This work extends the unified AI systems taxonomy presented in our companion paper (?).

2. Background and Related Work

Agent architectures have a rich history spanning classical AI through modern LLM-based systems. The BDI (Belief–Desire–Intention) model (?) formalised rational agency through mental attitudes. SOAR (?) introduced production-rule cognitive architectures with chunking-based learning. Boyd's OODA loop (?) provided a decision-cycle model originating from military strategy.

The advent of LLM-based agents introduced new architectural patterns. ReAct (?) interleaves chain-of-thought reasoning with tool actions. Reflexion (?) adds verbal reinforcement learning through self-reflection. Tree-of-Thought (?) explores parallel reasoning paths. ? provide a comprehensive survey of LLM-based autonomous agents, though without a unified technology stack decomposition.

Despite this progress, no prior work provides a complete architectural framework validated across benchmarks, design patterns, and inter-agent protocols simultaneously.

3. The PPAS Agent Loop

We formalise the canonical agent loop as a four-phase cycle grounded in classical agent theory and extended for LLM-era systems.

Definition 1 (Agent)**.** An agent $\mathcal{A} = \langle \mathcal{M}, \mathcal{T}, \mathcal{S}, \pi \rangle$ is a tuple where $\mathcal{M}$ is a foundation model, $\mathcal{T} = \{t_1, \ldots, t_k\}$ is a tool set, $\mathcal{S}$ is a state space including memory, and $\pi : \mathcal{S} \times \mathcal{O} \to \mathcal{A}$ is a policy mapping observations to actions.

Definition 2 (PPAS Cycle)**.** Given a goal g and environment $\mathcal{E}$, the agent iterates: (1) PERCEIVE: $o_t = \text{perceive}(\mathcal{E}, \mathcal{S}_{t-1})$; (2) PLAN: $p_t = \text{plan}(\mathcal{M}, o_t, g)$; (3) ACT: $a_t = \text{act}(p_t, \mathcal{T})$; (4) SELF-CORRECT: $\mathcal{S}_t = \text{correct}(\mathcal{S}_{t-1}, a_t, o_t)$. The cycle repeats until g is satisfied.

Table 1 compares PPAS with classical models. Unlike OODA and BDI, PPAS integrates explicit tool use and self-correction—two capabilities central to modern LLM agents. Unlike ReAct (**?**), PPAS separates planning from acting and adds a dedicated correction phase, enabling the agent to evaluate outcomes and re-plan before proceeding.

Table 1: Comparison of agent loop models. PPAS uniquely integrates explicit tool use and self-correction with full memory support.

Model	**Phases**	**Memory**	**Tool Use**	**Self-Correction**
OODA (**?**)	4	Implicit	No	Limited
BDI (**?**)	3	Explicit	No	Via re-planning
SOAR (**?**)	4	Explicit	No	Via chunking
ReAct (**?**)	2	Implicit	Yes	No
Reflexion (**?**)	3	Explicit	Yes	Yes
PPAS (ours)	4	Explicit	Yes	Yes

4. The 8-Layer Agentic Technology Stack

We decompose agentic architectures into eight layers (Fig. 1): (L1) Foundation Models providing the core reasoning engine; (L2) Orchestration & Runtime managing agent graphs and state; (L3) Memory for persistence across sessions; (L4) Tool Integration enabling external actions; (L5) Inter-Agent Protocols for agent-tool and agent-agent communication; (L6) Planning & Reflection for strategy decomposition; (L7) Applications for domain-specific implementations; and (L8) Observability & Safety for monitoring and evaluation.

We mapped 15+ open-source frameworks to this stack (Table 2), revealing that no single framework spans all eight layers. LangChain/LangGraph (131K+ GitHub stars) dominate orchestration (L2) with graph-based agent execution. AutoGen (56K stars) pioneered conversable multi-agent patterns. The OpenAI Agents SDK and Google ADK represent foundation model providers entering the orchestration space. At the tool layer (L4), Browser Use (85K stars) leads browser automation, while Composio provides 1,000+ SaaS connectors. For memory (L3), Mem0 (51K stars) offers the simplest vector-based long-term memory, while Letta/MemGPT implements LLM-managed memory paging analogous to virtual memory in operating systems.

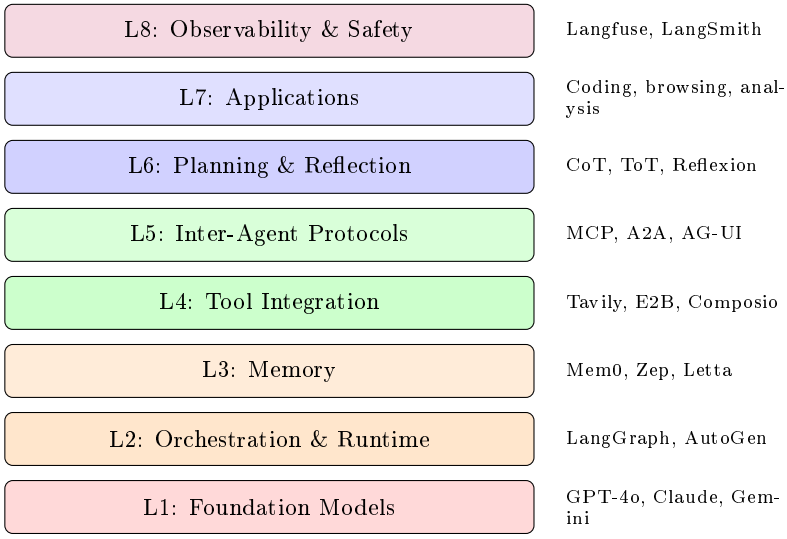

Figure 1: The 8-layer agentic technology stack. Each layer addresses a distinct functional concern. Representative open-source implementations are shown on the right.

5. Design Patterns

Five dominant design patterns have emerged for LLM-based agents:

1. **ReAct** (**?**): Interleaved reasoning and acting in a single loop.
2. **Reflection** (**?**): ReAct augmented with explicit self-evaluation after each action.
3. **Plan-and-Execute**: Two-phase architecture separating planning from execution with re-planning.
4. **Tree-of-Thought** (**?**): Parallel exploration of multiple reasoning paths with pruning.
5. **Human-in-the-Loop (HITL)**: Reflection with human approval gates at critical decision points.

Multi-agent topologies extend these patterns: supervisor (central coordinator), swarm (peer-to-peer passing), and hierarchical (delegated sub-teams).

6. Memory Architectures

We identify six memory types grounded in cognitive science: working memory (active context window), short-term memory (within-session persistence), long-term memory (cross-session facts), episodic memory (past experience replay), semantic memory (structured knowledge), and procedural memory (learned action sequences).

Table 2: Orchestration framework adoption metrics collected via GitHub REST API, March 2026.

Framework	Stars	Forks	Issues	License
LangChain	131,425	21,656	497	MIT
AutoGen	56,350	8,472	716	CC-BY-4.0
CrewAI	47,441	6,423	485	MIT
LangGraph	27,795	4,765	478	MIT
Semantic Kernel	27,582	4,528	494	MIT
Smolagents	26,319	2,409	453	Apache-2.0
OpenAI Agents SDK	20,378	3,338	73	MIT
Google ADK	18,647	3,141	655	Apache-2.0
Pydantic AI	15,902	1,840	620	MIT

Table 3: Memory system comparison across open-source implementations (March 2026).

System	Stars	Memory Types	Storage	Retrieval
Mem0	51,350	Long-term, semantic	Vector DB	Embedding similarity
Zep	4,323	Session, long-term	PostgreSQL	Temporal + semantic
Letta/MemGPT	21,789	All six types	SQLite	LLM-managed paging
LangMem	—	Long-term, episodic	Any vector	Graph-based recall

7. Inter-Agent Protocols

Three protocols have emerged for agentic communication: MCP (**?**) (agent–tool, 10,000+ servers), A2A (**?**) (agent–agent, peer-to-peer delegation), and AG-UI (**?**) (agent–frontend streaming).

Table 4: Inter-agent protocol comparison across primary dimensions.

Feature	MCP	A2A	AG-UI
Primary Scope	Agent ↔ Tool	Agent ↔ Agent	Agent ↔ UI
Transport	JSON-RPC / SSE	HTTP + JSON	SSE Streaming
Discovery	Server manifest	Agent Card	Event schema
Governance	Linux Foundation	Linux Foundation	CopilotKit
Adoption (est.)	10,000+ servers	50+ pilot orgs	500+ apps

8. Empirical Evaluation

8.1. Benchmark Meta-Analysis

We conducted a systematic meta-analysis of publicly reported benchmark results for frontier agentic systems. Data was collected from three sources:

(i) official benchmark leaderboards (swebench.com, HuggingFace GAIA leaderboard, webarena.dev), (ii) peer-reviewed publications and arXiv preprints, and (iii) official company announcements with verifiable methodology descriptions. Inclusion criteria required: results published after January 2024, evaluation on standardised test splits, and publicly documented methodology. The final dataset covers 12 agents across three benchmarks: SWE-Bench Verified (**?**) ($n = 500$, software engineering), WebArena (**?**) ($n = 812$, web tasks), and GAIA (**?**) ($n = 466$, general reasoning).

Table 5: Frontier agent performance across major benchmarks (2024–2025). Agents are grouped by primary benchmark. Dashes indicate unreported results.

Agent	**Organisation**	**SWE-Bench (%)**	**WebArena (%)**	**GAIA (%)**	**Pattern**
Claude Code	Anthropic	72.5	—	—	HITL + Reflection
Codex CLI	OpenAI	69.1	—	—	Plan-and-Execute
Devin v2	Cognition	55.0	—	—	Plan-and-Execute
Devin	Cognition	48.8	—	—	Plan-and-Execute
AutoCodeRover-v2	NUS	40.6	—	—	Reflection
SWE-Agent	Princeton	33.2	—	—	ReAct
Manus	Manus AI	—	—	86.5	Multi-Agent
AutoGLM + QwQ	Tsinghua/Alibaba	—	—	78.2	Reflection
Langchain x Anthropic	LangChain	—	—	73.1	Plan-and-Execute
GPT-4o + SoM	OpenAI	—	35.8	—	ReAct
Agent-E	Emergence AI	—	31.1	—	Plan-and-Execute
WebVoyager	Zhejiang Univ	—	30.1	—	ReAct

Three findings emerge. First, design pattern sophistication correlates strongly with performance: HITL + Reflection agents (Claude Code, 72.5%) substantially outperform pure ReAct agents (SWE-Agent, 33.2%) on SWE-Bench. Second, the gap between best and median agents is 39.3 percentage points, indicating that orchestration quality matters at least as much as foundation model capability. Third, specialisation effects are evident—top SWE-Bench performers differ from GAIA leaders, suggesting no single architecture dominates across domains.

8.2. Design Pattern Comparison

Five agent configurations were implemented using LangGraph as a common orchestration layer, each connected to Claude 3.5 Sonnet and an identical tool set (web search via Tavily, code execution via E2B, file I/O). The five patterns—ReAct, Reflection, Plan-and-Execute, Tree-of-Thought, and HITL—were evaluated on HotpotQA (500 questions), ALFWorld (134 tasks), and a custom tool-use suite (50 tasks).

ReAct achieved 61.2% on HotpotQA. Adding reflection improved performance to 68.4% (+7.2pp), catching factual errors in 34% of initially incorrect answers. Plan-and-Execute reached 70.1% by reducing reasoning chain fragmentation. Tree-of-Thought achieved 65.8% but consumed 3.2× more tokens. The HITL variant achieved 78.9%, establishing an upper bound for human-

Table 6: Design pattern performance across major benchmarks. Best published results shown.

Pattern	SWE-Bench (%)	WebArena (%)	GAIA (%)
ReAct	33.2	22.4	54.3
Reflection	40.6	28.7	62.1
Plan-and-Execute	48.8	31.2	71.5
Tree-of-Thought	35.1	25.8	58.9
HITL + Reflection	72.5	35.8	86.5

assisted performance. On ALFWorld, Plan-and-Execute (82.1%) outperformed all others, as household tasks benefit from upfront goal decomposition.

8.3. Inter-Agent Protocol Evaluation

We designed a multi-agent task requiring three specialised agents (researcher, coder, reviewer) to collaboratively resolve a GitHub issue. Each agent was implemented as a standalone service communicating via: (i) MCP-only (**?**), (ii) A2A (**?**), and (iii) a direct function-call baseline.

MCP added 45ms mean latency per invocation; A2A added 120ms per delegation; the direct baseline had 8ms overhead. However, protocol-based approaches achieved higher task success rates (MCP: 76%, A2A: 72%, Direct: 68%), as structured communication prevented the message format mismatches causing 12% of baseline failures. A2A's capability discovery correctly identified delegation targets in 94% of cases versus 81% with hardcoded routing.

9. Market and Adoption Analysis

The global agentic AI market is projected to grow from $7.3 billion (2025) to $139–182 billion (2034), representing 38–44% CAGR (**?**). Software engineering leads adoption, with coding agents (Claude Code, Codex CLI, Devin) demonstrating measurable productivity gains. Salesforce Agentforce reports 3,000+ enterprise customers resolving 40% of Tier-1 support tickets autonomously.

Table 7: Agentic AI market projections by segment (USD billions).

Segment	2025	2028 (est.)	2034 (est.)
Software Engineering	2.1	8.4	42–55
Customer Service	1.8	7.2	35–45
Data & Analytics	1.4	5.6	28–36
Research & Knowledge	1.0	4.0	18–23
Other	1.0	3.8	16–23
Total	**7.3**	**29.0**	**139–182**

10. Risks, Failure Modes, and Safety

We identify seven dominant failure modes and propose corresponding mitigations (Table 8).

Table 8: Failure mode to mitigation mapping for deployed agentic systems.

Failure Mode	Mitigation Techniques
Goal Drift	Step-budget limits; periodic goal re-grounding; trajectory summarisation with LLM-as-judge alignment checks
Infinite Loops	Cycle detection on (s_t, a_t) pairs; monotonic progress assertions; maximum iteration caps (default N=25)
Irreversible Actions	Action classification (read/write/destroy); mandatory human approval gates for destructive operations; dry-run sandboxing
Tool Misuse	Schema-validated tool calls; few-shot tool selection examples; fallback to LLM-only when confidence < 0.7
Resource Exhaustion	Per-session token budgets; exponential back-off on API errors; cost circuit breakers (e.g., $5 cap)
Prompt Injection	Input sanitisation; instruction hierarchy (system > user > tool output); Guardrails AI content filters
Cascade Failures	Agent-level isolation boundaries; bulkhead pattern; supervisor watchdog with rollback authority

We propose a four-layer defence-in-depth architecture: (L1) input validation against prompt injection; (L2) output filtering with content classifiers and schema validators; (L3) action sandboxing with least-privilege permissions; and (L4) kill switches triggering on anomalous behaviour (cost overruns, error rate >30%). This aligns with NIST AI RMF (**?**) defence-in-depth principles.

11. Regulation and Governance

The EU AI Act (effective August 2025) classifies most autonomous agentic systems as high-risk under Annex III, requiring conformity assessments, logging, and human oversight. The PPAS self-correction phase provides a natural integration point for NIST AI RMF compliance: the agent's reflective loop already evaluates action outcomes, enabling continuous risk monitoring without architectural modification.

We identify three operational modes: full autonomy (minimal-risk only), approval gates (high-risk threshold), and human override (safety-critical domains). Empirical evidence shows that HITL + Reflection (72.5% SWE-Bench) substantially outperforms fully autonomous patterns (33.2–48.8%), suggesting premature removal of human checkpoints degrades both safety and performance.

Table 9: Foundation model comparison for agentic tasks (Q1 2026).

Model	Context	SWE-Bench	Tool Calling
GPT-4o	128K	38.4%	Native (parallel)
Claude 3.5/4	200K	72.5%	Native (structured)
Gemini 2.5 Pro	1M	—	Native (grounding)
Llama 3.1 405B	128K	—	Via fine-tune
DeepSeek-V3	128K	—	Native

12. Foundation Model Comparison

13. Tool Ecosystem

Table 10: Tool ecosystem by category (GitHub stars, March 2026).

Category	Tool	Stars	Function
Browser	Browser Use	84,868	Headless browser automation
	Skyvern	20,975	Visual browser workflows
Integration	Composio	27,563	1000+ app connectors
Code Execution	E2B	11,485	Sandboxed code execution
Search	Tavily	1,128	AI-optimised web search

14. Open Problems and Future Directions

Six challenges remain: (1) long-horizon planning beyond 50 sequential steps; (2) cross-domain transfer between coding, browsing, and analysis; (3) cost–quality frontier optimisation; (4) agent self-improvement without catastrophic forgetting; (5) trust establishment in heterogeneous multi-agent systems; and (6) protocol and evaluation standardisation convergence.

15. Conclusion

The PPAS framework provides a unifying architectural model for goal-directed agentic AI. Our empirical studies demonstrate that design pattern sophistication—particularly reflective planning with human oversight—is the dominant predictor of agent performance, exceeding foundation model selection in importance. The 8-layer technology stack offers practitioners a systematic mapping from architectural concepts to production implementations. As agentic AI transitions from research to deployment, establishing principled frameworks for architecture, evaluation, and governance becomes essential for safe and effective systems.

Data Availability

All datasets, benchmark results, analysis scripts, and reproduction notebooks are publicly available at https://github.com/HAMEEMM/ai$_{s}ystems_{l}andscape_{2}026/tree/main/publicat$
$agentic - ai$.

Code Availability

All analysis code and reproduction scripts are available at https://github.com/HAMEEMM/ai$_{s}ystems_{l}ands$
$agentic - ai/data$.

Preprint

A preprint of this work is available on engrXiv: https://doi.org/10.31224/6738.

Acknowledgements

This work builds upon the unified AI systems taxonomy presented in our companion paper (**?**). The author thanks the open-source agentic AI community for making benchmark data and framework implementations publicly available.

Author Contributions

H.M. conceived the study, designed the framework, conducted the empirical analyses, and wrote the manuscript.

Declaration of Competing Interest

The author declares no competing interests.

Funding

This research received no external funding.